Pthreads: simulación transferencia de calor

Computación de altas prestaciones

Máster universitario ingeniería informática



VNiVERSiDAD D SALAMANCA

Autores

Lur Perona Bzik

Alejandro Martín Martín

Fecha

Noviembre de 2025

Profesor

Javier Blanco

Tabla de contenidos

1 Introducción **1**

2 Descripción del programa codificado **1**

2.1 Programa secuencial 1

2.2 Programa multi-hilo 2

3 Entorno de pruebas **2**

4 Resultados de las pruebas **3**

5 Rendimiento **4**

5.1 Speedup 4

5.2 Eficiencia 5

5.2.1 Ejemplo 7

5.3 Otras métricas 7

6 Conclusiones **7**

Apéndice **8**

A Tablas de resultados **8**

Referencias **11**

Listado de figuras

1 Procesadores e hilos del servidor. 3
2 Comparación de tiempos según número de hilos y tamaño de matriz. 4
3 Comparación de tiempos (solo 16, 24, 32 y 40 hilos). 4
4 Comparación de Speedup (N = 1000). 5
5 Comparación de speedup general. 5
6 Comparación de tiempos extendido para 24, 32 y 40 hilos. 5
7 Eficiencia del paralelismo. 6

Listado de tablas

1 Tiempos medios (en segundos) según número de hilos y tamaño de la matriz . . . 3
2 Speedup respecto a la ejecución secuencial (1 hilo). 4
3 Eficiencia del algoritmo para distintos tamaños de matriz. 6
4 Instantes de tiempo discreto según la dimensión de la matriz. 6
5 Tiempos de ejecución para 1 hilo 8
6 Tiempos de ejecución para 2 hilos 8
7 Tiempos de ejecución para 4 hilos 8
8 Tiempos de ejecución para 8 hilos 8
9 Tiempos de ejecución para 16 hilos 9
10 Tiempos de ejecución para 24 hilos 9
11 Tiempos de ejecución para 32 hilos 9
12 Tiempos de ejecución para 40 hilos 9
13 Tiempos de ejecución extendidos para 24 hilos 10
14 Tiempos de ejecución extendidos para 32 hilos 10
15 Tiempos de ejecución extendidos para 40 hilos 10

Listado de ejemplos de código

1 Pseudocódigo de programa secuencial 1
2 Pseudocódigo de programa paralelo 2

1. Introducción

En el presente informe se analizan y comparan los resultados temporales obtenidos mediante la ejecución de un programa de simulación de transferencia de calor, empleando múltiples hilos para una ejecución paralela. Se emplea un sistema de **memoria compartida**. Las operaciones para los cálculos de temperaturas se realizan empleando números en punto flotante de precisión simple.
Parte de la información mostrada en este informe ha sido consultada y/o adaptada de (Universidad de Salamanca, 2025).

2. Descripción del programa codificado

En esta sección se comentan las características más relevantes de los programas codificados. El primero es un programa sin paralelismo (todas las operaciones son realizadas por el mismo proceso), mientras que el segundo emplea múltiples hilos. En ambos casos la matriz de temperaturas se inicializa a valor 20 en todas las celdas a excepción de:

- $M_{1,1}$ se inicializa a 0.
- $M_{n,m}$ se inicializa a 100.

2.1. Programa secuencial

Se ha optado por codificar un programa diferente en este caso (en lugar de emplear un único hilo en la versión multi-hilo) para evitar la penalización temporal de las primitivas de sincronización. De esta forma tenemos:

$$T_{\text{secuencial}} = \text{Tiempo de ejecución secuencial}$$

```
1    MatrizActual[n][m],
2    MatrizAnterior[n][m]
3
4    mientras (NO FinPrograma) {
5        dt = 0
6        para cada elemento en MatrizActual:
7            MatrizActual[i][j] = calcularTemperatura(MatrizActual[i][j], MatrizAnterior)
8            dt += abs(MatrizActual[i][j] - MatrizAnterior[i][j])
9        fin para
10
11       intercambiarPunteros(matrizAnterior, matrizActual)
12       FinPrograma = comprobar(dt)
13   }
14
15   guardarResultados()
```

Código 1: Pseudocódigo de programa secuencial

Donde la condición de finalización es:

$$\sum_{i=1}^{n}\sum_{j=1}^{m} |\text{MatrizActual}_{i,j} - \text{MatrizAnterior}_{i,j}| < 10^{-6}$$

Es decir, se alcanza un estado estacionario.

2.2. Programa multi-hilo

Como primitivas de sincronización, se ha utilizado:

- **1 Semáforo** para que cada hilo actualice la variable compartida que marca la condición de finalización de forma segura.
- **1 Mutex y 1 variable de condición** para crear una barrera que sincronización que garantice que todos los hilos finalizan sus operaciones correspondientes a un mismo instante de tiempo antes de continuar con la siguiente iteración. De este modo, ningún hilo avanza hasta que todos han alcanzado el mismo punto de ejecución.

$$T_{\text{paralelo}} = T_{\text{computación}} + T_{\text{comunicación}} + T_{\text{espera}}$$

```
MatrizActual[n][m],
MatrizAnterior[n][m],
nhilos,

CODIGO PARALELO
filasDelHilo = calcularFilas(nhilos, n)

mientras (NO FinPrograma) {
    localDT = 0

    para cada elemento en filasDelHilo:
        MatrizActual[i][j] = calcularTemperatura(MatrizActual[i][j], MatrizAnterior)
        localDT += abs(MatrizActual[i][j] - MatrizAnterior[i][j])
    fin para

    semWait()
    globalDT += localDT
    semPost()

    bloquearBarrera()
    si (ULTIMO HILO EN LLEGAR):
        intercambiarPunteros(matrizAnterior, matrizActual)
        FinPrograma = comprobar(globalDT)
        globalDT = 0
        reanudarBarrera()
    fin si

    si no (ULTIMO HILO EN LLEGAR):
        Esperar()
    fin sino
}
FIN CODIGO PARALELO

guardarResultados()
```

Código 2: Pseudocódigo de programa paralelo

Donde la condición de finalización es la misma que en el ejemplo secuencial[2.1].

3. Entorno de pruebas

Para la realización de las pruebas experimentales y posterior análisis de los resultados obtenidos, se ha empleado el servidor de prácticas **vdia3.fis.usal.es**, cuyas características son las siguientes

("Servidor vdía 3", 2025):

- Dell PowerEdge R440
- 2 x Xeon Silver 4210R (20 Cores-40 Threads)
- 128GB de RAM DDR4-3200. (4x32GB Hynix HMA84GR7CJR4N-XN)
- SSD 512GB – RAID 1
- Sistema Linux Debian 10 – 64bits

También podemos comprobar cuántos hilos admite el procesador ejecutándose simultáneamente utilizando el comando *lscpu*.

```
<vdia3>/home/i3519833$ lscpu
Arquitectura:                        x86_64
modo(s) de operación de las CPUs:    32-bit, 64-bit
Orden de los bytes:                  Little Endian
Tamaños de las direcciones:          46 bits physical, 48 bits virtual
CPU(s):                              40
Lista de la(s) CPU(s) en línea:      0-39
Hilo(s) de procesamiento por núcleo: 2
```

Figura 1: Procesadores e hilos del servidor.

Como se observa en la figura 1, se puede ejecutar un máximo de **40 hilos** de forma simultánea:

- 40 CPUs lógicas.
- 2 hilos de procesamiento por núcleos.

Por tanto, 20 núcleos físicos y 40 hilos hardware totales. Este será el límite empleado en la realización de las pruebas experimentales.

4. Resultados de las pruebas

En esta sección se evalúan los resultados obtenidos de las ejecuciones con diferentes tamaños de matriz y números de hilos. En todos los casos se han considerado matrices cuadradas. Para maximizar la fiabilidad de los resultados obtenidos, cada caso se ha ejecutado 5 veces y se trabajará con la media temporal obtenida. Pueden verse los resultados desglosados en el apéndice A.

Hilos	**N=100**	**N=500**	**N=750**	**N=1000**
1	3.4104	81.0702	181.5155	324.2330
2	1.8176	40.2785	90.1637	158.7615
4	1.0624	20.9299	46.9997	83.3607
8	0.8278	11.0775	24.3756	43.3023
16	1.1966	7.1092	13.8142	23.0468
24	1.8603	7.8848	16.0324	27.5000
32	2.7377	6.8677	13.1435	21.7862
40	3.6166	10.1978	15.1011	18.1111

Tabla 1: Tiempos medios (en segundos) según número de hilos y tamaño de la matriz

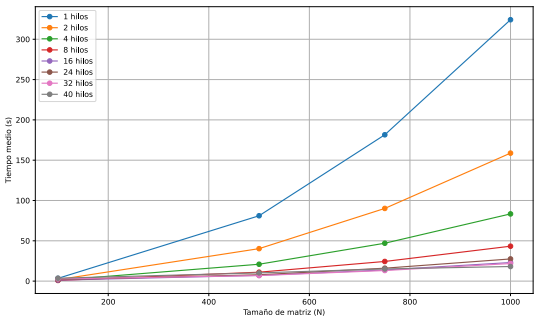


Figura 2: Comparación de tiempos según número de hilos y tamaño de matriz.

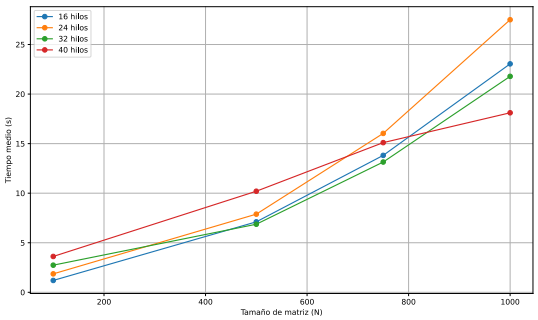


Figura 3: Comparación de tiempos (solo 16, 24, 32 y 40 hilos).

5. Rendimiento

En esta sección se analizan métricas de rendimiento en base a los resultados obtenidos.

5.1. Speedup

El *speedup* es una métrica que mide la relación entre el tiempo de ejecución en un sólo procesador y la ejecución multiprocesador.

$$S = \frac{T_{\text{secuencial}}}{T_{\text{paralelo}}}$$

En la tabla 2 se muestra el speedup para múltiples números de hilos y tamaños de matrices.

Hilos	N=100	N=500	N=750	N=1000
2	1.8763	2.0127	2.0132	2.0423
4	3.2101	3.8734	3.8621	3.8895
8	4.1198	7.3185	7.4466	7.4877
16	2.8501	11.4036	13.1398	14.0685
24	1.8333	10.2818	11.3218	11.7903
32	1.2457	11.8046	13.8103	14.8825
40	0.9430	7.9498	12.0200	17.9024

Tabla 2: Speedup respecto a la ejecución secuencial (1 hilo).

En la tabla anterior tenemos 3 valores donde $S < P$, es decir, tenemos un Speedup superlineal. Esto puede deberse principalmente a dos motivos:

- El algoritmo secuencial no es óptimo.
- Las características hardware ponen el algoritmo secuencial en desventaja.

Hay que tener en cuenta que el porcentaje de superlinealidad es menor al 1 %, lo cuál es una desviación muy pequeña que se puede atribuir a ruido de medidas.

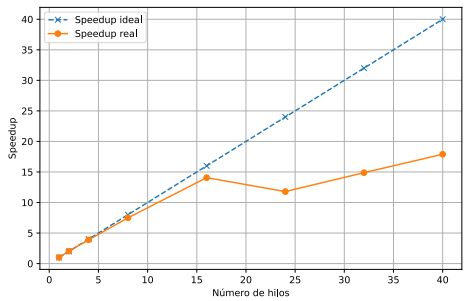


Figura 4: Comparación de Speedup (N = 1000).

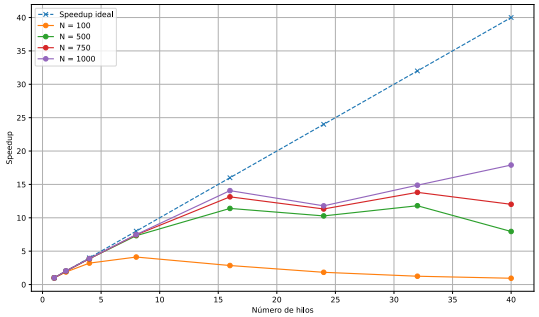


Figura 5: Comparación de speedup general.

5.2. Eficiencia

La eficiencia es la fracción de tiempo que los hilos realizan trabajo útil.

$$E = \frac{S}{p} = \frac{T_{\text{secuencial}}}{p \cdot T_{\text{paralelo}}}$$

Esta métrica tiende a:

- **Aumentar** cuando el el tamaño del problema **aumenta**.
- **Disminuir** cuando el número de procesos **aumenta**.

Si nos fijamos en la Figura 3, podemos apreciar que para tamaños pequeños de N (tamaño del problema), el tiempo de ejecución es superior en para un número menor de hilos. En cambio, conforme aumentamos N y el tamaño del problema aumenta, también lo hace la eficiencia. Puede apreciarse este comportamiento en la Figura 6.

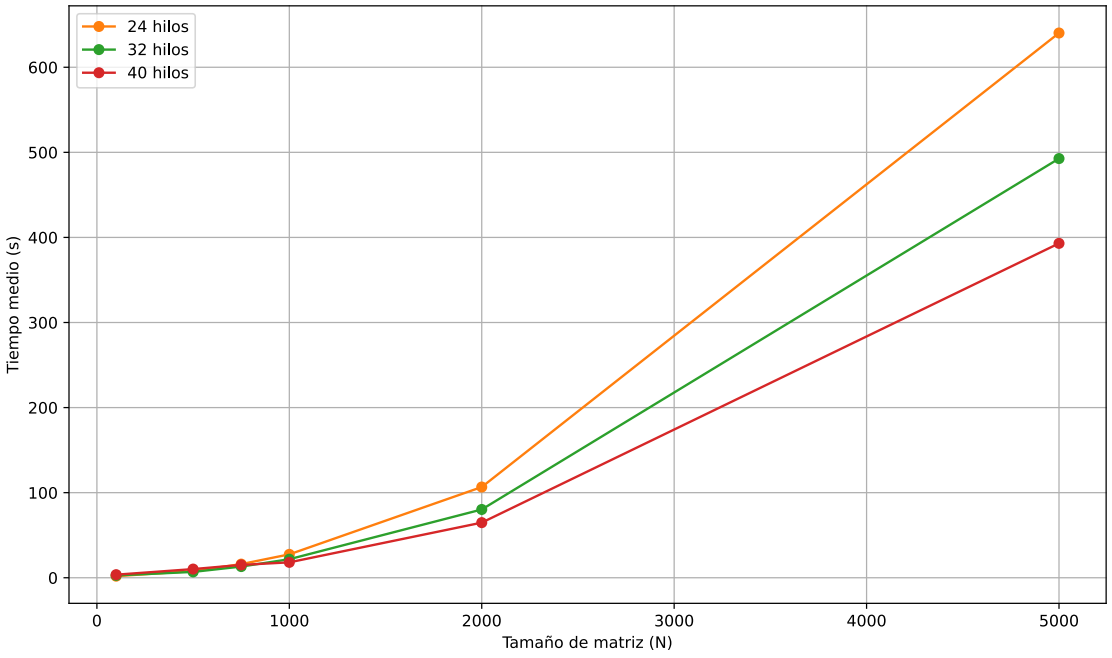


Figura 6: Comparación de tiempos extendido para 24, 32 y 40 hilos.

Hilos	N = 100	N = 500	N = 750	N = 1000
2	0.9382	1.0064	1.0066	1.0211
4	0.8025	0.9684	0.9655	0.9724
8	0.5150	0.9148	0.9308	0.9360
16	0.1781	0.7127	0.8212	0.8793
24	0.0764	0.4284	0.4717	0.4913
32	0.0389	0.3689	0.4316	0.4651
40	0.0236	0.1987	0.3005	0.4476

Tabla 3: Eficiencia del algoritmo para distintos tamaños de matriz.

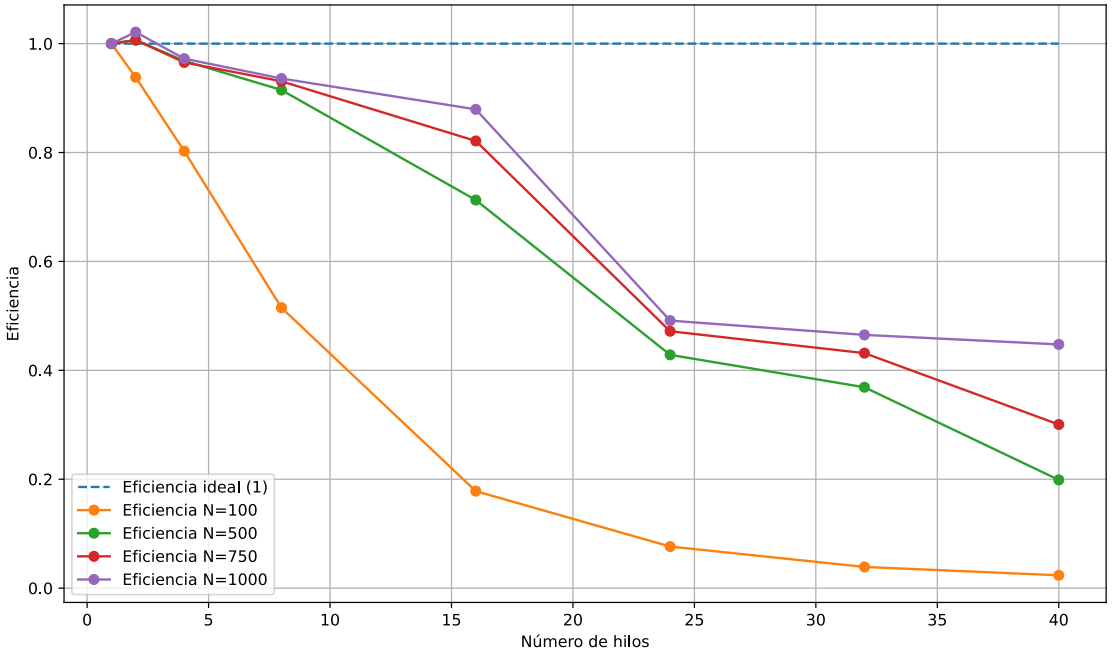


Figura 7: Eficiencia del paralelismo.

En computación paralela un sistema es escalable si mantiene constante la eficiencia al aumentar el número de procesadores aumentando también el tamaño del problema. En este caso, el tamaño del problema depende de la dimensión de la matriz y del número de instantes de tiempo discreto calculados hasta llegar a un estado estacionario.

$$\text{Tamaño del problema} \sim (\text{dimensión de la matriz})^2 \times \text{número de instantes de tiempo}$$

Dimensión de la matriz	Instantes de tiempo discreto
100	17459
500	16959
750	16959
1000	16959

Tabla 4: Instantes de tiempo discreto según la dimensión de la matriz.

Si el sistema es escalable, deberíamos obtener la misma eficiencia al duplicar el tamaño del problema y duplicar el número de hilos. Sin embargo, aumentar la dimensión de la matriz, aumenta el tamaño del problema de forma cuadrática, no lineal.

5.2.1. Ejemplo

El tamaño del problema P en este caso es:

$$P \sim N^2 \cdot T$$

Si duplicamos la dimensión de la matriz de 500 a 1000:

$$\frac{P_{\text{nuevo}}}{P_{\text{antiguo}}} = \frac{1000^2}{500^2} = 4$$

Es decir, el tamaño del problema aumenta **4 veces**. Para mantener eficiencia constante, idealmente también se aumenta el número de hilos 4 veces. Analizando los resultados de la tabla 3 observamos:

- Para $N = 500$ e $Hilos = 2$: $Eficiencia = 1.0064$
- Para $N = 1000$ e $Hilos = 8$: $Eficiencia = 0.9360$

Otro caso:

- Para $N = 500$ e $Hilos = 4$: $Eficiencia = 0.9684$
- Para $N = 1000$ e $Hilos = 16$: $Eficiencia = 0.8793$

Otro caso:

- Para $N = 500$ e $Hilos = 8$: $Eficiencia = 0.9148$
- Para $N = 1000$ e $Hilos = 32$: $Eficiencia = 0.4651$

Este ejemplo muestra que la escalabilidad se mantiene aproximadamente para pocos hilos, pero a medida que se aumenta demasiado, la eficiencia disminuye significativamente.

5.3. Otras métricas

Existen otras métricas para evaluar el rendimiento de un programa paralelo como el **coste** total de cómputo, es decir, la cantidad total de tiempo de procesador gastado para completar la tarea.

$$C = p \cdot T_{\text{paralelo}}$$

O el **overhead** que mide el tiempo de trabajo "extra" que se gasta debido a paralelización.

$$T_{\text{overhead}} = p \cdot T_{\text{paralelo}} - T_{\text{secuencial}} = C - T_s$$

6. Conclusiones

En este informe se ha realizado un análisis teórico y empírico del rendimiento del programa paralelo. Como principales conclusiones se obtienen:

- A medida que aumenta el número de hilos, los mecanismos de sincronización introducen mayor overhead, lo que reduce la *eficiencia* del paralelismo.
- A medida que aumenta el tamaño del problema, la eficiencia mejora, ya que el coste relativo de la sincronización disminuye frente al cómputo total.

Apéndice

A. Tablas de resultados

Nhilos	Nfil	Ncol	Tiempos	Media
1	100	100	3.411303 3.414885 3.412173 3.398166 3.415651	3.4104
1	500	500	81.152463 81.166187 80.944176 80.988550 81.099559	81.0702
1	750	750	181.167505 181.630953 181.319863 181.251177 182.207988	181.5155
1	1000	1000	325.265384 325.445903 322.270554 323.201327 324.982000	324.2330

Tabla 5: Tiempos de ejecución para 1 hilo

Nhilos	Nfil	Ncol	Tiempos	Media
2	100	100	1.810529 1.815243 1.808209 1.832405 1.821855	1.8176
2	500	500	40.505589 40.391796 40.618285 39.305677 40.571045	40.2785
2	750	750	90.269440 90.110582 90.317294 89.850673 90.270464	90.1637
2	1000	1000	157.370708 159.897996 158.341329 157.918840 160.278606	158.7615

Tabla 6: Tiempos de ejecución para 2 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
4	100	100	1.062461 1.068199 1.039712 1.067426 1.074141	1.0624
4	500	500	21.101317 20.678984 21.175523 21.263932 20.429946	20.9299
4	750	750	47.020373 47.024512 46.891735 47.059177 47.002877	46.9997
4	1000	1000	83.463848 83.087294 83.359679 83.460219 83.432621	83.3607

Tabla 7: Tiempos de ejecución para 4 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
8	100	100	0.858636 0.840474 0.840920 0.769323 0.829728	0.8278
8	500	500	11.084514 11.050140 11.070920 11.099064 11.082725	11.0775
8	750	750	24.183463 24.475747 24.411507 24.379662 24.427841	24.3756
8	1000	1000	43.262399 43.720478 43.231268 43.123253 43.173873	43.3023

Tabla 8: Tiempos de ejecución para 8 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
16	100	100	1.192517 1.182777 1.239566 1.178387 1.189842	1.1966
16	500	500	7.464415 7.123108 6.927613 7.019921 7.011021	7.1092
16	750	750	13.945058 13.779426 13.622320 13.725844 13.998459	13.8142
16	1000	1000	23.062797 23.046139 23.163797 23.005058 22.956177	23.0468

Tabla 9: Tiempos de ejecución para 16 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
24	100	100	1.847302 1.850927 1.853913 1.851644 1.897905	1.8603
24	500	500	7.909710 7.841926 7.916852 7.870033 7.885448	7.8848
24	750	750	16.065069 15.948341 16.128525 16.101858 15.918161	16.0324
24	1000	1000	27.304458 27.333808 27.468352 27.629601 27.763750	27.5000

Tabla 10: Tiempos de ejecución para 24 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
32	100	100	2.711239 2.734243 2.830134 2.700577 2.712221	2.7377
32	500	500	6.897999 6.934725 6.873243 6.825923 6.806374	6.8677
32	750	750	13.118559 13.173870 13.002431 13.362075 13.060599	13.1435
32	1000	1000	21.632266 21.767793 21.695572 21.655164 22.180245	21.7862

Tabla 11: Tiempos de ejecución para 32 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
40	100	100	3.574044 3.505158 3.572026 3.708618 3.723226	3.6166
40	500	500	10.179029 10.280814 10.191701 10.182637 10.154600	10.1978
40	750	750	18.362314 18.237799 16.317528 11.259118 11.328847	15.1011
40	1000	1000	18.246046 18.128585 18.063338 18.042662 18.074989	18.1111

Tabla 12: Tiempos de ejecución para 40 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
24	2000	2000	106.735660 106.411772 106.058803 107.056938 106.897355	106.6321
24	5000	5000	642.233364 639.467318 639.754211 639.303813 640.554019	640.2625

Tabla 13: Tiempos de ejecución extendidos para 24 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
32	2000	2000	80.095193 80.455062 80.197749 80.344156 80.147222	80.2479
32	5000	5000	491.775450 492.572161 493.825169 493.087728 492.164716	492.6850

Tabla 14: Tiempos de ejecución extendidos para 32 hilos

Nhilos	Nfil	Ncol	Tiempos	Media
40	2000	2000	64.715412 64.809987 64.824388 64.879583 64.684976	64.7829
40	5000	5000	393.015324 393.315399 392.974078 392.865040 392.915310	393.0170

Tabla 15: Tiempos de ejecución extendidos para 40 hilos

Referencias

Servidor vdía 3 [Disponible en línea.]. (2025). https://studium25.usal.es/pluginfile.php/416402/mod_resource/content/4/servidor_vdia3.pdf

Universidad de Salamanca. (2025). *Bases de Computación Paralela* (Documento docente / Apuntes) (Accedido: 10 noviembre 2025). Universidad de Salamanca. https://studium25.usal.es/pluginfile.php/415610/mod_resource/content/2/Bases%20de%20computaci%C3%B3n%20paralela.pdf